Feasibility Report: PII Detection POC with Dual Validation Pipelines

Title: PII Detection POC with Amazon Comprehend-Bedrock and Azure Language-OpenAI

Prepared by: Nithin Nayak
Reviewed by: Prakhyath Rai,Suraj Nagraj

Version: v1.0

Table of Contents

Section	Page
1. Introduction	3
1.1 Problem Statement	3
1.2 Objectives	3
2. Literature Study	4
2.1 AWS Comprehend	4
2.2 AWS Bedrock	5
2.3 Azure AI Language Service	6
2.4 Azure OpenAI	7
3. Infrastructure & Software Requirement Specification	8
3.1 Cloud Requirements	8
3.2 Software Requirements	8
4. Architecture & Methodology	8
4.1 System Architecture	8
4.1.1 AWS-Based Pipeline	9
4.1.2 Azure-Based Pipeline	10
4.2 Methodology	11
5. Results and Analysis	13
5.1 Results	13
5.2 Comparative Analysis	18
5.3 Cost Analysis	19
6. Risks and Limitations	20
7. Conclusion	20

1. Introduction

1.1 Problem Statement

In customer or user-generated content (e.g., support tickets, feedback forms, chats), Personally Identifiable Information (PII) often appears inadvertently. Exposing PII leads to legal, ethical, and compliance risks (e.g., GDPR, HIPAA). Manual redaction is time-consuming and error-prone, while traditional regex-based masking lacks context sensitivity and adaptability to diverse text formats.

Current Gaps:

- Inconsistent detection of PII across structured/unstructured data
- Lack of context-aware masking
- Scalability and multi-cloud compatibility
- No validation or second-level assurance for false positives/negatives

1.2 Objectives

Technical Objectives:

- Implement a dual-layered PII detection and masking pipeline using:
    - AWS Comprehend (primary) + AWS Bedrock (validation)
    - Azure AI Language (primary) + OpenAI (validation)
- Evaluate effectiveness, latency, and cost of each pipeline

Business Objectives:

- Ensure compliance with privacy regulations (GDPR, HIPAA)
- Improve customer trust by securing personal data
- Reduce manual effort and errors in data handling workflows

2. Literature Study

2.1 AWS Comprehend

Overview

Amazon Comprehend is a fully managed Natural Language Processing (NLP) service that leverages machine learning to extract insights, relationships, and sensitive information from unstructured text. It includes a pre-trained Personally Identifiable Information (PII) detection API, eliminating the need for custom model development in many compliance and security use cases.

Key Features:

- **Entity Recognition:** Identifies common entities such as names, organizations, locations, dates, and numerical identifiers.
- **PII Detection:** Supports detection of structured PII types, including:
  - **Personal Identifiers:** Social Security Numbers (SSN), passport numbers, driver's license IDs
  - **Contact Information:** Email addresses, phone numbers, IP addresses
  - **Financial Data:** Credit card numbers, bank account details
- **Confidence Scoring:** Assigns a probability score (0–1) for each detected entity, allowing threshold-based filtering.

Supported Languages:

- Primarily optimized for English, with limited support for other major languages (e.g., Spanish, French, German).

Limitations:

1. **Domain-Specific PII:** Cannot detect custom or industry-specific PII (e.g., proprietary employee IDs, internal codes).

2. **Contextual Understanding:** Lacks semantic reasoning (e.g., distinguishing between a real SSN and a fictional one in narrative text).
3. **Redaction Handling:** Only identifies PII; redaction must be implemented separately.

2.2 AWS Bedrock

Overview

AWS Bedrock is a managed service providing API access to foundational large language models (LLMs) such as Anthropic Claude, Amazon Titan, and Meta Llama 3. It enables programmatic PII validation through prompt engineering.

Use Case in PII Detection:

- Acts as a **validation layer** by querying LLMs with contextual prompts (e.g., *"Does this text contain sensitive information? List any PII with reasoning."*).
- Augments rule-based systems by detecting ambiguous or novel PII patterns (e.g., disguised contact info like *"user[at]domain.com"*).

Strengths:

- **Context Awareness:** LLMs infer intent (e.g., distinguishing medical records vs. fictional narratives).
- **Adaptability:** Supports dynamic fine-tuning via prompts (e.g., industry-specific PII definitions).
- **Multilingual Support:** Inherits LLM capabilities for non-English text.

Limitations:

1. **Cost:** Per-token pricing incurs higher operational expenses than static APIs.

2. **Latency:** LLM inference is slower (~500ms–2s) compared to AWS Comprehend (~200ms).
3. **Non-Determinism:** Outputs may vary across model versions or prompts.

2.3 Azure AI Language Service

Overview

Azure AI Language Service (part of Microsoft's Cognitive Services) offers a Text Analytics API with deep-learning-based PII detection and redaction. It is designed for enterprise compliance (e.g., GDPR, HIPAA).

Key Features:

- **Language-Specific Detection:** Optimized models for 20+ languages (e.g., Spanish IDs, Japanese names).
- **Redaction Flexibility:** Configurable actions for identified PII:
  - **Masking:** Replaces characters (e.g., ***-***-1234).
  - **Replacement:** Substitutes with labels (e.g., [PHONE]).
  - **Removal:** Deletes PII segments entirely.
- **Threshold Control:** Users can set minimum confidence scores (e.g., discard detections below 0.9).
- **PII Categories:** Covers:
  - **Healthcare:** Patient IDs, ICD codes
  - **Finance:** SWIFT codes, tax IDs
  - **Government:** National IDs, passport numbers

Limitations:

1. **Regional Compliance:** Redaction rules may not align with local regulations (e.g., Asia-Pacific data privacy laws).

2. **Model Updates:** Changes to pre-trained models (e.g., new PII types) require API version updates.

2.4 Azure OpenAI

Azure OpenAI is a cloud-based service offered by Microsoft in collaboration with OpenAI, providing enterprise-grade access to advanced **large language models (LLMs)** such as **GPT-4, GPT-3.5, and Embeddings models** within a secure, compliant, and scalable Azure infrastructure. It enables businesses to integrate cutting-edge AI capabilities into applications while adhering to enterprise security and privacy requirements.

Capabilities:

- **Dispute Resolution:** Flags false positives/negatives from rule-based systems with natural language explanations (e.g., *"This 9-digit number is a product SKU, not an SSN"*).
- **Complex Detection:** Identifies implied PII (e.g., *"My mom's birthday is 12/31/1990"* infers a date of birth).

Limitations:

1. **Cost Prohibitive:** GPT-4 costs ~$0.06/1K tokens, making large-scale processing expensive.
2. **Ambiguity Risk:** Over-reliance on LLM reasoning may introduce new false positives.

3. Infrastructure & Software Requirement Specification

3.1 Cloud Requirements

- AWS Account
- AWS Bedrock
- Azure Subscription
- Azure OpenAI

3.2 Software Requirements

- **Programming Language**: Python 3.10+
- **Key Libraries/SDKs**:
  - boto3
  - langchain (prompt orchestration with AWS Bedrock, Azure OpenAI)
  - openai (for Azure OpenAI calls if needed)
  - pandas

4. Architecture & Methodology

4.1 System Architecture

Two parallel PII masking pipelines are implemented—one using AWS services and the other using Azure services. Both pipelines follow a similar flow but leverage different vendor-specific components for detection and validation

4.1.1 AWS-Based PII Detection and Redaction Pipeline

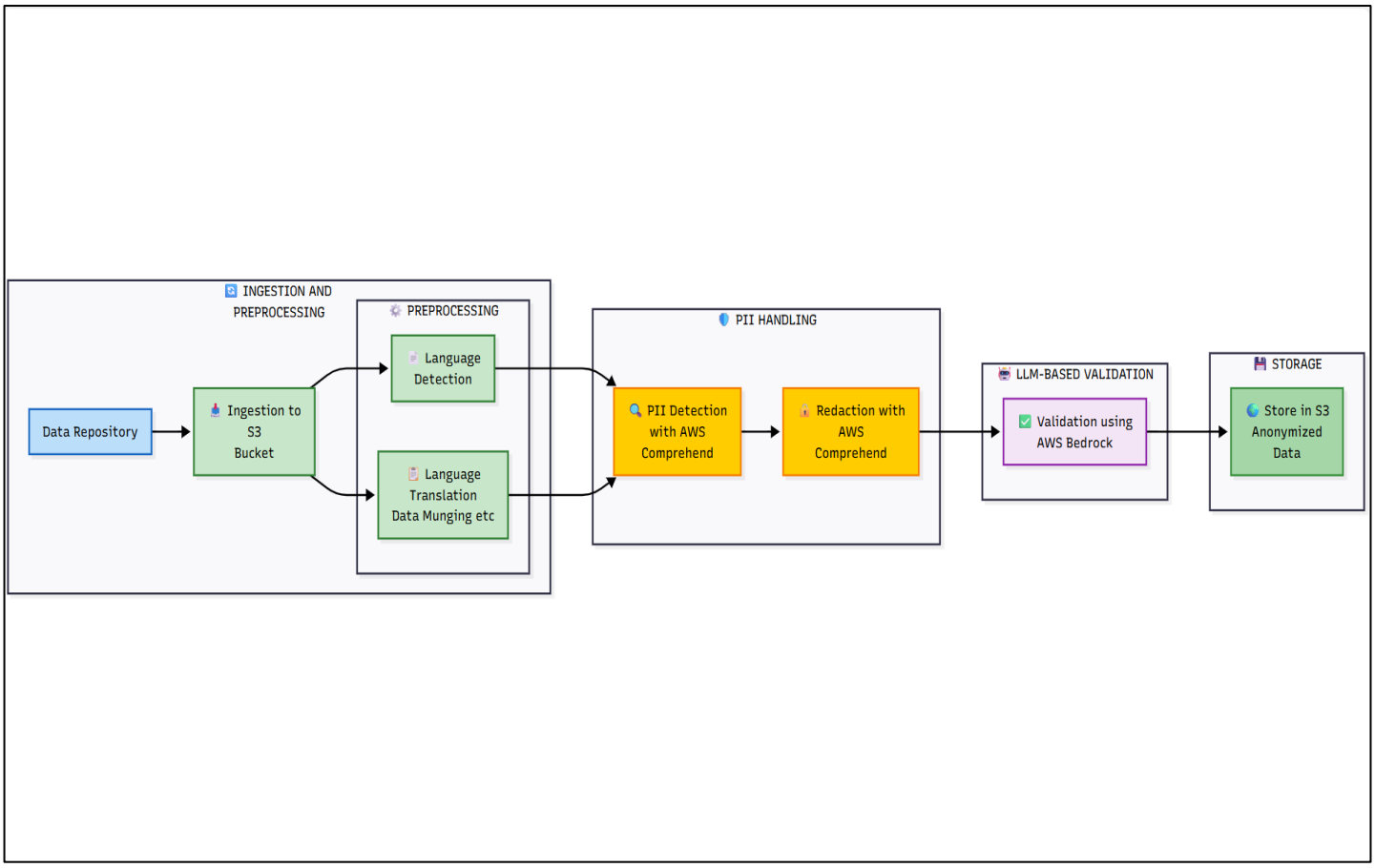

Figure 4.1.1: AWS PII Detection and Redaction Architecture

1. **Input Source**
   - Amazon S3: Raw text files (e.g., logs, CSVs, documents) are ingested from a predefined S3 bucket.
2. **Trigger & Preprocessing**
   - Lambda/Fargate: Retrieves the file, performs:
     - Language Detection: Ensures correct PII handling for multilingual content.
3. **Primary PII Detection**
   - Amazon Comprehend: Analyses text and classifies PII entities (names, emails, SSNs, etc.).
4. **LLM-Based Validation**

- AWS Bedrock (Claude/Titan): Validates Comprehends output via prompt engineering (e.g., *"Verify if this is a real credit card number: [entity]"*).

5. **Redaction & Output**
   - Masking Rules: Replaces PII with placeholders (e.g., [REDACTED]) or pseudonyms.
   - Processed File Storage: Redacted output is saved to a separate S3 bucket.
6. **Audit & Monitoring**
   - CloudWatch Logs: Tracks execution metrics, errors, and PII detection stats.
   - S3 Access Logs: Records file access for compliance auditing.

4.1.2 Azure-Based PII Detection and Redaction Pipeline

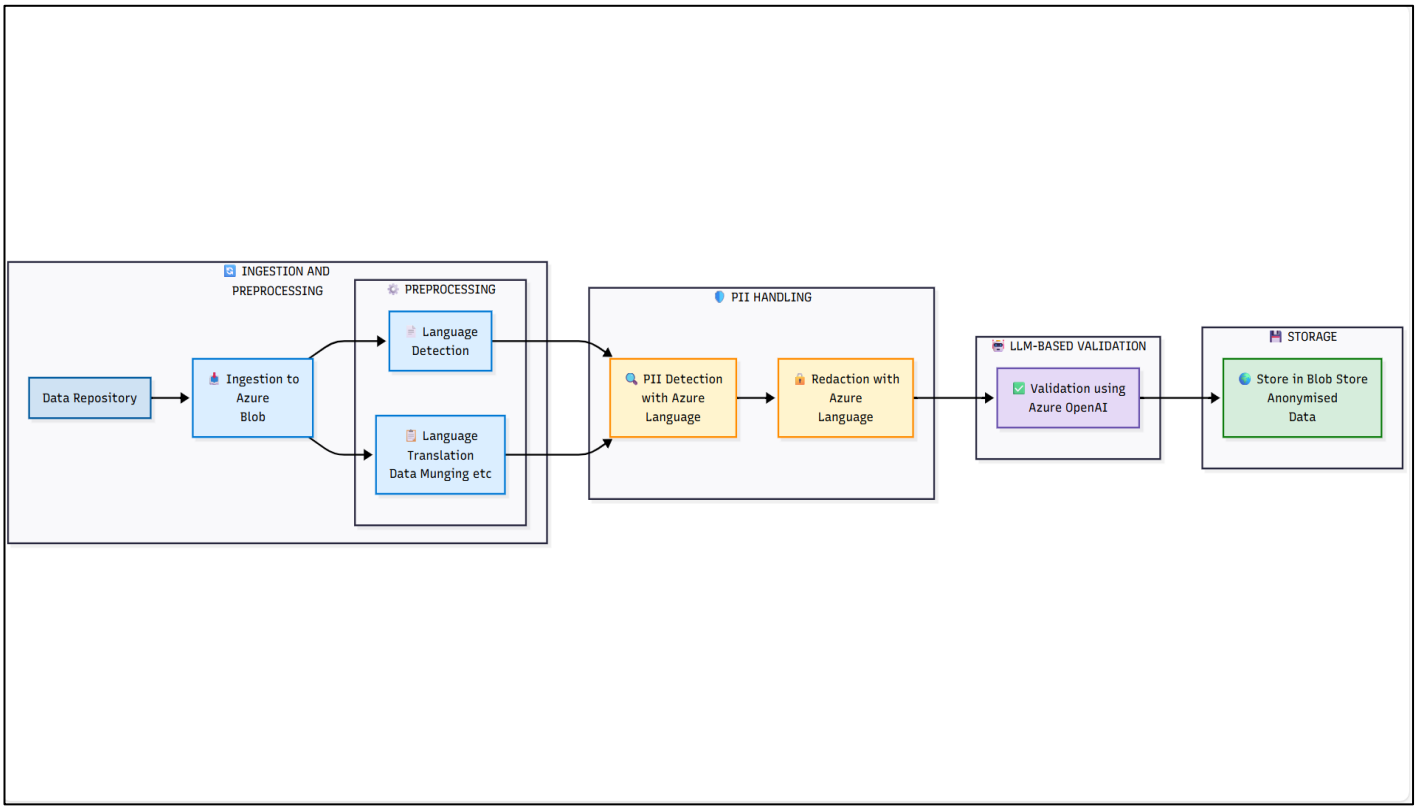

Figure 4.1.2: Azure PII Detection and Redaction Architecture

1. **Input Source**
    - **Azure Blob Storage**: Raw text files (e.g., logs, CSVs, documents) are ingested from a specified Blob container.
2. **Trigger & Preprocessing**
    - **Azure Functions / Web Jobs**: Fetches the file and performs:
        - **Language Detection**: Ensures PII handling aligns with the text's language.
3. **Primary PII Detection**
    - **Azure AI Language (Text Analytics)**: Identifies and classifies PII (emails, SSNs, locations, etc.) using pre-trained models.
4. **LLM-Based Validation**
    - **Azure OpenAI (GPT-4)**: Validates detected PII via contextual prompts (e.g., *"Is this a valid SSN? [entity] – Respond with True/False"*). Reduces false positives/negatives.
5. **Redaction & Output**
    - **Dynamic Masking**: Applies rules (e.g., hashing, partial redaction, or placeholder replacement like [REDACTED]).
    - **Processed File Storage**: Cleaned output is saved to a separate Blob container with access controls.
6. **Audit & Monitoring**
    - **Azure Monitor / Application Insights**: Tracks latency, errors, and PII detection accuracy.
    - **Log Analytics**: Centralized logging for compliance audits (e.g., GDPR, CCPA).

4.2 Methodology

Pipeline Workflow Steps:

Data Acquisition:

- Raw customer service text data ingested from APIs, databases, or data lakes. For PoC excel file considered as input.

Preprocessing:

- Language normalization, sentence tokenization, encoding fixes.

Primary PII Detection:

- **Cloud-Native Services**:
  - **AWS**: **Amazon Comprehend** (pre-trained PII entity recognition for emails, SSNs, etc.).
  - **Azure**: **Azure Text Analytics for Language** (detects PII like names, addresses).
- **Output**: Structured JSON with entity types, confidence scores, and text offsets.

Redaction Strategy:

- Replace each detected PII entity with a masked version (e.g., <EMAIL>, <PHONE>)
- Maintain original indices for audit logs

PII Validation via LLMs:

- **Purpose**: Reduce false positives/negatives by cross-verifying detected PII with contextual LLM analysis.
  - **AWS**: **Bedrock** (e.g., Anthropic Claude or Llama 2) validates entities via prompts like: *"Is '[detected_text]' truly a [PII type] in this context: '[surrounding_sentence]'?"*
  - **Azure**: **Azure OpenAI** (GPT-4) performs similar validation with adjustable confidence thresholds.
- **Outcome**: Only high-confidence PII proceeds to redaction.

Output & Storage:

- Both pipelines store redacted data, logs, and audit files securely in cloud object storage
- Integrated monitoring to detect failures or false negatives

5. Results and Analysis

5.1 Results

For this proof of concept (PoC), an Excel file containing dummy PII data was used and tested with AWS and Azure-based services. PII detection was intentionally applied only to three columns: 'first_name', 'last_name', and 'comment'. The 'email' field was excluded from detection to evaluate whether the LLM-powered validation layer would correctly identify and redact it. The results of this evaluation are summarized below:

AWS Pipeline Results

The figure 5.1: a depicts the input file in which sensitive data is required to be detected and redacted. Whereas the figure 5.1: b depicts the file anonymized by the comprehend service and figure 5.1:c depicts the file validated and non-redacted pii data redacted by bedrock service.The figure5.1: d depicts the llm summarizing briefly pii coverage report.

Note: In Figure 5.2b, several entries in the 'first_name' and 'last_name' columns remain un-anonymized, raising concerns about the accuracy of Amazon Comprehend.

emp_id	first_name	last_name	email	designation	comment
E1001	Emily	Johnson	emily.johnson@example.com	Product Manager	DOB: 1990-01-01
E1002	Emily	Williams	emily.williams@example.com	Product Manager	Emergency contact: John Doe, 999-888-7777
E1003	Chris	Brown	chris.brown@example.com	Software Engineer	Requires special access.
E1004	John	Davis	john.davis@example.com	QA Engineer	Emergency contact: John Doe, 999-888-7777
E1005	Chris	Brown	chris.brown@example.com	HR Executive	Lives at 123 Main Street, Springfield.
E1006	Jessica	Johnson	jessica.johnson@example.com	Product Manager	Passport number: A12345678
E1007	Jessica	Garcia	jessica.garcia@example.com	Product Manager	Requires special access.
E1008	Chris	Miller	chris.miller@example.com	Product Manager	Requires special access.
E1009	Laura	Jones	laura.jones@example.com	Data Analyst	Requires special access.
E1010	Chris	Johnson	chris.johnson@example.com	QA Engineer	SSN: 123-45-6789
E1011	Jane	Davis	jane.davis@example.com	Software Engineer	DOB: 1990-01-01
E1012	Chris	Jones	chris.jones@example.com	Product Manager	SSN: 123-45-6789
E1013	Emily	Davis	emily.davis@example.com	Data Analyst	Emergency contact: John Doe, 999-888-7777
E1014	David	Garcia	david.garcia@example.com	QA Engineer	Emergency contact: John Doe, 999-888-7777
E1015	John	Davis	john.davis@example.com	Software Engineer	Lives at 123 Main Street, Springfield.
E1016	Chris	Brown	chris.brown@example.com	QA Engineer	Emergency contact: John Doe, 999-888-7777
E1017	Emily	Garcia	emily.garcia@example.com	QA Engineer	SSN: 123-45-6789
E1018	Michael	Smith	michael.smith@example.com	Product Manager	Email: jane.doe@personalmail.com
E1019	Jane	Johnson	jane.johnson@example.com	Software Engineer	N/A
E1020	Laura	Brown	laura.brown@example.com	Software Engineer	N/A

Figure 5.1a: Input Excel

emp_id	first_name	last_name	email	designation	comment
E1001	[NAME]	[NAME]	emily.johnson@example.	Product Manager	DOB: [DATE_TIME]
E1002	[NAME]	[NAME]	emily.williams@example.	Product Manager	Emergency contact: [NAME], [PHONE]
E1003	[NAME]	Brown	chris.brown@example.co	Software Engineer	Requires special access.
E1004	[NAME]	[NAME]	john.davis@example.com	QA Engineer	Emergency contact: [NAME], [PHONE]
E1005	[NAME]	Brown	chris.brown@example.co	HR Executive	Lives at [ADDRESS].
E1006	[NAME]	[NAME]	jessica.johnson@example	Product Manager	Passport number: [PASSPORT_NUMBER]
E1007	[NAME]	[NAME]	jessica.garcia@example.c	Product Manager	Requires special access.
E1008	[NAME]	[NAME]	chris.miller@example.cor	Product Manager	Requires special access.
E1009	[NAME]	[NAME]	laura.jones@example.cor	Data Analyst	Requires special access.
E1010	[NAME]	[NAME]	chris.johnson@example.c	QA Engineer	SSN: [SSN]
E1011	Jane	[NAME]	jane.davis@example.com	Software Engineer	DOB: [DATE_TIME]
E1012	[NAME]	[NAME]	chris.jones@example.con	Product Manager	SSN: [SSN]
E1013	[NAME]	[NAME]	emily.davis@example.cor	Data Analyst	Emergency contact: [NAME], [PHONE]
E1014	[NAME]	[NAME]	david.garcia@example.co	QA Engineer	Emergency contact: [NAME], [PHONE]
E1015	[NAME]	[NAME]	john.davis@example.com	Software Engineer	Lives at [ADDRESS].
E1016	[NAME]	Brown	chris.brown@example.co	QA Engineer	Emergency contact: [NAME], [PHONE]
E1017	[NAME]	[NAME]	emily.garcia@example.co	QA Engineer	SSN: [SSN]
E1018	[NAME]	Smith	michael.smith@example.	Product Manager	Email: [EMAIL]
E1019	Jane	[NAME]	jane.johnson@example.c	Software Engineer	
E1020	[NAME]	Brown	laura.brown@example.cc	Software Engineer	

Figure 5.1b: AWS Comprehend Redacted O/p

emp_id	first_name	last_name	email	designation	comment
E1001	[NAME]	[NAME]	[EMAIL]	Product Manager	DOB: [DATE_TIME]
E1002	[NAME]	[NAME]	[EMAIL]	Product Manager	Emergency contact: [NAME], [PHONE]
E1003	[NAME]	[NAME]	[EMAIL]	Software Engineer	Requires special access.
E1004	[NAME]	[NAME]	[EMAIL]	QA Engineer	Emergency contact: [NAME], [PHONE]
E1005	[NAME]	[NAME]	[EMAIL]	HR Executive	Lives at [ADDRESS].
E1006	[NAME]	[NAME]	[EMAIL]	Product Manager	Passport number: [PASSPORT_NUMBER]
E1007	[NAME]	[NAME]	[EMAIL]	Product Manager	Requires special access.
E1008	[NAME]	[NAME]	[EMAIL]	Product Manager	Requires special access.
E1009	[NAME]	[NAME]	[EMAIL]	Data Analyst	Requires special access.
E1010	[NAME]	[NAME]	[EMAIL]	QA Engineer	SSN: [SSN]
E1011	[NAME]	[NAME]	[EMAIL]	Software Engineer	DOB: [DATE_TIME]
E1012	[NAME]	[NAME]	[EMAIL]	Product Manager	SSN: [SSN]
E1013	[NAME]	[NAME]	[EMAIL]	Data Analyst	Emergency contact: [NAME], [PHONE]
E1014	[NAME]	[NAME]	[EMAIL]	QA Engineer	Emergency contact: [NAME], [PHONE]
E1015	[NAME]	[NAME]	[EMAIL]	Software Engineer	Lives at [ADDRESS].
E1016	[NAME]	[NAME]	[EMAIL]	QA Engineer	Emergency contact: [NAME], [PHONE]
E1017	[NAME]	[NAME]	[EMAIL]	QA Engineer	SSN: [SSN]
E1018	[NAME]	[NAME]	[EMAIL]	Product Manager	Email: [EMAIL]
E1019	[NAME]	[NAME]	[EMAIL]	Software Engineer	
E1020	[NAME]	[NAME]	[EMAIL]	Software Engineer	

Figure 5.1c: AWS Bedrock Validated O/p


```
{
  "comment": "6 PII fields masked in 'first_name', 'last_name', 'email', 'comment' fields across 20 records.",
  "data": [
    {
      "emp_id": "E1001",
      "first_name": "[NAME]",
      "last_name": "[NAME]",
      "email": "[EMAIL]",
      "designation": "Product Manager",
      "comment": "DOB: [DATE_TIME]"
    },
    {
      "emp_id": "E1002",
      "first_name": "[NAME]",
      "last_name": "[NAME]",
      "email": "[EMAIL]",
      "designation": "Product Manager",
      "comment": "Emergency contact: [NAME], [PHONE]"
    },
    {
      "emp_id": "E1003",
      "first_name": "[NAME]",
      "last_name": "[NAME]",
      "email": "[EMAIL]",
      "designation": "Software Engineer",
      "comment": "Requires special access."
    },
    {
      "emp_id": "E1004",
      "first_name": "[NAME]",
      "last_name": "[NAME]",
      "email": "[EMAIL]",
      "designation": "QA Engineer",
      "comment": "Emergency contact: [NAME], [PHONE]"
    },
    {
      "emp_id": "E1005",
      "first_name": "[NAME]",
```


Figure 5.1d: AWS Bedrock Summary Of Validation O/p

Azure Pipeline Results

The figure 5.1:e depicts the input file in which sensitive data is required to be detected and redacted. Whereas the figure 5.1: f depicts the file anonymized by the azure language service and figure 5.1:g depicts the file

validated and non-redacted pii data redacted by azure openai service. The figure5.1: h depicts the llm summarizing briefly pii coverage report.

emp_id	first_name	last_name	email	designation	comment
E1001	Emily	Johnson	emily.johnson@example.com	Product Manager	DOB: 1990-01-01
E1002	Emily	Williams	emily.williams@example.com	Product Manager	Emergency contact: John Doe, 999-888-7777
E1003	Chris	Brown	chris.brown@example.com	Software Engineer	Requires special access.
E1004	John	Davis	john.davis@example.com	QA Engineer	Emergency contact: John Doe, 999-888-7777
E1005	Chris	Brown	chris.brown@example.com	HR Executive	Lives at 123 Main Street, Springfield.
E1006	Jessica	Johnson	jessica.johnson@example.com	Product Manager	Passport number: A12345678
E1007	Jessica	Garcia	jessica.garcia@example.com	Product Manager	Requires special access.
E1008	Chris	Miller	chris.miller@example.com	Product Manager	Requires special access.
E1009	Laura	Jones	laura.jones@example.com	Data Analyst	Requires special access.
E1010	Chris	Johnson	chris.johnson@example.com	QA Engineer	SSN: 123-45-6789
E1011	Jane	Davis	jane.davis@example.com	Software Engineer	DOB: 1990-01-01
E1012	Chris	Jones	chris.jones@example.com	Product Manager	SSN: 123-45-6789
E1013	Emily	Davis	emily.davis@example.com	Data Analyst	Emergency contact: John Doe, 999-888-7777
E1014	David	Garcia	david.garcia@example.com	QA Engineer	Emergency contact: John Doe, 999-888-7777
E1015	John	Davis	john.davis@example.com	Software Engineer	Lives at 123 Main Street, Springfield.
E1016	Chris	Brown	chris.brown@example.com	QA Engineer	Emergency contact: John Doe, 999-888-7777
E1017	Emily	Garcia	emily.garcia@example.com	QA Engineer	SSN: 123-45-6789
E1018	Michael	Smith	michael.smith@example.com	Product Manager	Email: jane.doe@personalmail.com
E1019	Jane	Johnson	jane.johnson@example.com	Software Engineer	N/A
E1020	Laura	Brown	laura.brown@example.com	Software Engineer	N/A

Figure 5.1e: Input Excel

emp_id	first_name	last_name	email	designation	comment
E1001	[Person]	[Person]	emily.johnson@ex	Product Manager	DOB: [DateTime]
E1002	[Person]	[Person]	emily.williams@ex	Product Manager	Emergency contact: [Person], [PhoneNumber]
E1003	[Person]	[Person]	chris.brown@exar	Software Engineer	Requires special access.
E1004	[Person]	[Person]	john.davis@exam	QA Engineer	Emergency contact: [Person], [PhoneNumber]
E1005	[Person]	[Person]	chris.brown@exar	HR Executive	Lives at [Address].
E1006	[Person]	[Person]	jessica.johnson@e	Product Manager	Passport number: [EUPassportNumber]
E1007	[Person]	[Person]	jessica.garcia@exa	Product Manager	Requires special access.
E1008	[Person]	[Person]	chris.miller@exar	Product Manager	Requires special access.
E1009	[Person]	[Person]	laura.jones@exar	Data Analyst	Requires special access.
E1010	[Person]	[Person]	chris.johnson@ex	QA Engineer	SSN: [USSocialSecurityNumber]
E1011	[Person]	[Person]	jane.davis@exam	Software Engineer	DOB: [DateTime]
E1012	[Person]	[Person]	chris.jones@exam	Product Manager	SSN: [USSocialSecurityNumber]
E1013	[Person]	[Person]	emily.davis@exar	Data Analyst	Emergency contact: [Person], [PhoneNumber]
E1014	[Person]	[Person]	david.garcia@exar	QA Engineer	Emergency contact: [Person], [PhoneNumber]
E1015	[Person]	[Person]	john.davis@exam	Software Engineer	Lives at [Address].
E1016	[Person]	[Person]	chris.brown@exar	QA Engineer	Emergency contact: [Person], [PhoneNumber]
E1017	[Person]	[Person]	emily.garcia@exar	QA Engineer	SSN: [USSocialSecurityNumber]
E1018	[Person]	[Person]	michael.smith@ex	Product Manager	Email: [Email]
E1019	[Person]	[Person]	jane.johnson@exa	Software Engineer	
E1020	[Person]	[Person]	laura.brown@exa	Software Engineer	

Figure 5.1f: Azure Language Redacted O/p

emp_id	first_name	last_name	email	designation	comment
E1001	[Person]	[Person]	[EMAIL]	Product Manager	DOB: [DateTime]
E1002	[Person]	[Person]	[EMAIL]	Product Manager	Emergency contact: [Person], [PhoneNumber]
E1003	[Person]	[Person]	[EMAIL]	Software Engineer	Requires special access.
E1004	[Person]	[Person]	[EMAIL]	QA Engineer	Emergency contact: [Person], [PhoneNumber]
E1005	[Person]	[Person]	[EMAIL]	HR Executive	Lives at [Address].
E1006	[Person]	[Person]	[EMAIL]	Product Manager	Passport number: [EUPassportNumber]
E1007	[Person]	[Person]	[EMAIL]	Product Manager	Requires special access.
E1008	[Person]	[Person]	[EMAIL]	Product Manager	Requires special access.
E1009	[Person]	[Person]	[EMAIL]	Data Analyst	Requires special access.
E1010	[Person]	[Person]	[EMAIL]	QA Engineer	SSN: [USSocialSecurityNumber]
E1011	[Person]	[Person]	[EMAIL]	Software Engineer	DOB: [DateTime]
E1012	[Person]	[Person]	[EMAIL]	Product Manager	SSN: [USSocialSecurityNumber]
E1013	[Person]	[Person]	[EMAIL]	Data Analyst	Emergency contact: [Person], [PhoneNumber]
E1014	[Person]	[Person]	[EMAIL]	QA Engineer	Emergency contact: [Person], [PhoneNumber]
E1015	[Person]	[Person]	[EMAIL]	Software Engineer	Lives at [Address].
E1016	[Person]	[Person]	[EMAIL]	QA Engineer	Emergency contact: [Person], [PhoneNumber]
E1017	[Person]	[Person]	[EMAIL]	QA Engineer	SSN: [USSocialSecurityNumber]
E1018	[Person]	[Person]	[EMAIL]	Product Manager	Email: [Email]
E1019	[Person]	[Person]	[EMAIL]	Software Engineer	
E1020	[Person]	[Person]	[EMAIL]	Software Engineer	

Figure 5.1g: Azure OpenAI Validated O/p


```
{"data": [{"emp_id": "E1001", "first_name": "[Person]", "last_name": "[Person]", "email": "[EMAIL]", "designation": "Product Manager", "comment": "DOB:
[DateTime]"}, {"emp_id": "E1002", "first_name": "[Person]", "last_name": "[Person]", "email": "[EMAIL]", "designation": "Product Manager", "comment":
"Emergency contact: [Person], [PhoneNumber]"}, {"emp_id": "E1003", "first_name": "[Person]", "last_name": "[Person]", "email": "[EMAIL]", "designation":
"Software Engineer", "comment": "Requires special access."}, {"emp_id": "E1004", "first_name": "[Person]", "last_name": "[Person]", "email": "[EMAIL]",
"designation": "QA Engineer", "comment": "Emergency contact: [Person], [PhoneNumber]"}, {"emp_id": "E1005", "first_name": "[Person]", "last_name": "[Person]",
"email": "[EMAIL]", "designation": "HR Executive", "comment": "Lives at [Address]."}, {"emp_id": "E1006", "first_name": "[Person]", "last_name": "[Person]",
"email": "[EMAIL]", "designation": "Product Manager", "comment": "Passport number: [EUPassportNumber]"}, {"emp_id": "E1007", "first_name": "[Person]",
"last_name": "[Person]", "email": "[EMAIL]", "designation": "Product Manager", "comment": "Requires special access."}, {"emp_id": "E1008", "first_name":
"[Person]", "last_name": "[Person]", "email": "[EMAIL]", "designation": "Product Manager", "comment": "Requires special access."}, {"emp_id": "E1009",
"first_name": "[Person]", "last_name": "[Person]", "email": "[EMAIL]", "designation": "Data Analyst", "comment": "Requires special access."}, {"emp_id":
"E1010", "first_name": "[Person]", "last_name": "[Person]", "email": "[EMAIL]", "designation": "QA Engineer", "comment": "SSN: [USSocialSecurityNumber]"},
{"emp_id": "E1011", "first_name": "[Person]", "last_name": "[Person]", "email": "[EMAIL]", "designation": "Software Engineer", "comment": "DOB: [DateTime]"},
{"emp_id": "E1012", "first_name": "[Person]", "last_name": "[Person]", "email": "[EMAIL]", "designation": "Product Manager", "comment": "SSN:
[USSocialSecurityNumber]"}, {"emp_id": "E1013", "first_name": "[Person]", "last_name": "[Person]", "email": "[EMAIL]", "designation": "Data Analyst",
"comment": "Emergency contact: [Person], [PhoneNumber]"}, {"emp_id": "E1014", "first_name": "[Person]", "last_name": "[Person]", "email": "[EMAIL]",
"designation": "QA Engineer", "comment": "Emergency contact: [Person], [PhoneNumber]"}, {"emp_id": "E1015", "first_name": "[Person]", "last_name": "[Person]",
"email": "[EMAIL]", "designation": "Software Engineer", "comment": "Lives at [Address]."}, {"emp_id": "E1016", "first_name": "[Person]", "last_name":
"[Person]", "email": "[EMAIL]", "designation": "QA Engineer", "comment": "Emergency contact: [Person], [PhoneNumber]"}, {"emp_id": "E1017", "first_name":
"[Person]", "last_name": "[Person]", "email": "[EMAIL]", "designation": "QA Engineer", "comment": "SSN: [USSocialSecurityNumber]"}, {"emp_id": "E1018",
"first_name": "[Person]", "last_name": "[Person]", "email": "[EMAIL]", "designation": "Product Manager", "comment": "Email: [Email]"}, {"emp_id": "E1019",
"first_name": "[Person]", "last_name": "[Person]", "email": "[EMAIL]", "designation": "Software Engineer", "comment": null}, {"emp_id": "E1020", "first_name"
"[Person]", "last_name": "[Person]", "email": "[EMAIL]", "designation": "Software Engineer", "comment": null}],

"comment": "20 PII fields masked in 'email' across all entries."}
```


Figure 5.1h: Azure OpenAI Validated O/p

5.2 Comparative Analysis

Feature	AWS Comprehend	Azure Language
Sentiment Analysis	Document & phrase-level	Document & sentence-level
Entity Recognition	Standard & custom entities	Standard, PII, healthcare, & custom entities (using CLU)
Language Detection	100+ languages	120+ languages
Custom Models	Custom entity recognition & classification	Custom NER & classification (Conversational Language Understanding - CLU)
PII Detection	Supports detection & redaction	Built-in PII detection for multiple languages
Healthcare NLP	Comprehend Medical (separate service)	Text Analytics for Health (integrated)
Pricing Model	Pay per 100 units of text processed	Pay per 1,000 transactions (API calls)

5.3 Cost Analysis

The following pricing models were evaluated for text processing and PII detection:

- AWS Bedrock (Claude 3 Haiku):
  - Input tokens: $0.00025 per 1,000
  - Output tokens: $0.00125 per 1,000
- Amazon Comprehend (PII detection):
  - Pricing: $0.0001 per 100 characters
- Azure Language Service (PII detection):
  - Pricing: $1 per 1,000 text records
- Azure OpenAI (GPT 4):
  - Input tokens: $0.06 per 1,000
  - Output tokens: $0.12 per 1,000

Key Takeaways

i. Azure OpenAI vs. Claude 3 Haiku:

   a. Azure OpenAI is significantly more expensive for both input and output tokens compared to Claude 3 Haiku on AWS Bedrock.

ii. Pricing Models:

   a. Azure bundles multiple features (e.g., PII detection, summarization) in its Language Service, which can be cost-effective for short documents (<5K characters) requiring multiple analyses.

b. AWS follows a per-feature pricing model, which may be cheaper for long documents or single-feature use cases.

iii. Optimal Use Cases:

a. Azure is more economical for short documents with bundled processing.

b. AWS may offer better cost efficiency for large-scale or single-feature workloads.

6. Risks and Limitations

- **Model Drift**: PII types evolve; LLM prompts may degrade without feedback loops
- **Latency**: Real-time applications may suffer due to LLM inference time
- **False Negatives**: Critical PII missed could cause compliance failures
- **Legal**: Need for proper logging and auditability for GDPR compliance

Mitigation Strategies:

- Periodic model validation
- Human-in-the-loop review for high-risk domains
- Audit trail and logging of each redaction step

7. Conclusion

This dual-pipeline approach ensures robust and accurate PII masking in dynamic, user-generated data. Azure and AWS pipelines complement each other, offering trade-offs between cost and precision. The system is scalable, compliant with regulatory standards, and provides high flexibility via LLM validation layers.

Next Steps:

- Pilot implementation on anonymized support tickets.
- Evaluate user feedback and audit results.
- Deploy into production with CI/CD pipelines and monitoring.

Appendix

Glossary

- **PII**: Personally Identifiable Information
- **LLM**: Large Language Model
- **Redaction**: The process of masking sensitive data
- **Prompt Engineering**: Customizing input prompts to guide LLM output

Dataset

- Excel file with dummy pii data (not publicly available).